CS3300 A - Introduction to Software Engineering

Lecture 17: Test Driven Development; Software Refactoring

Dr. Nimisha Roy ▸ nroy9@gatech.edu

Slides adapted from Alessandro Orso

Transition from Waterfall to Agile has made testing easier and more approachable

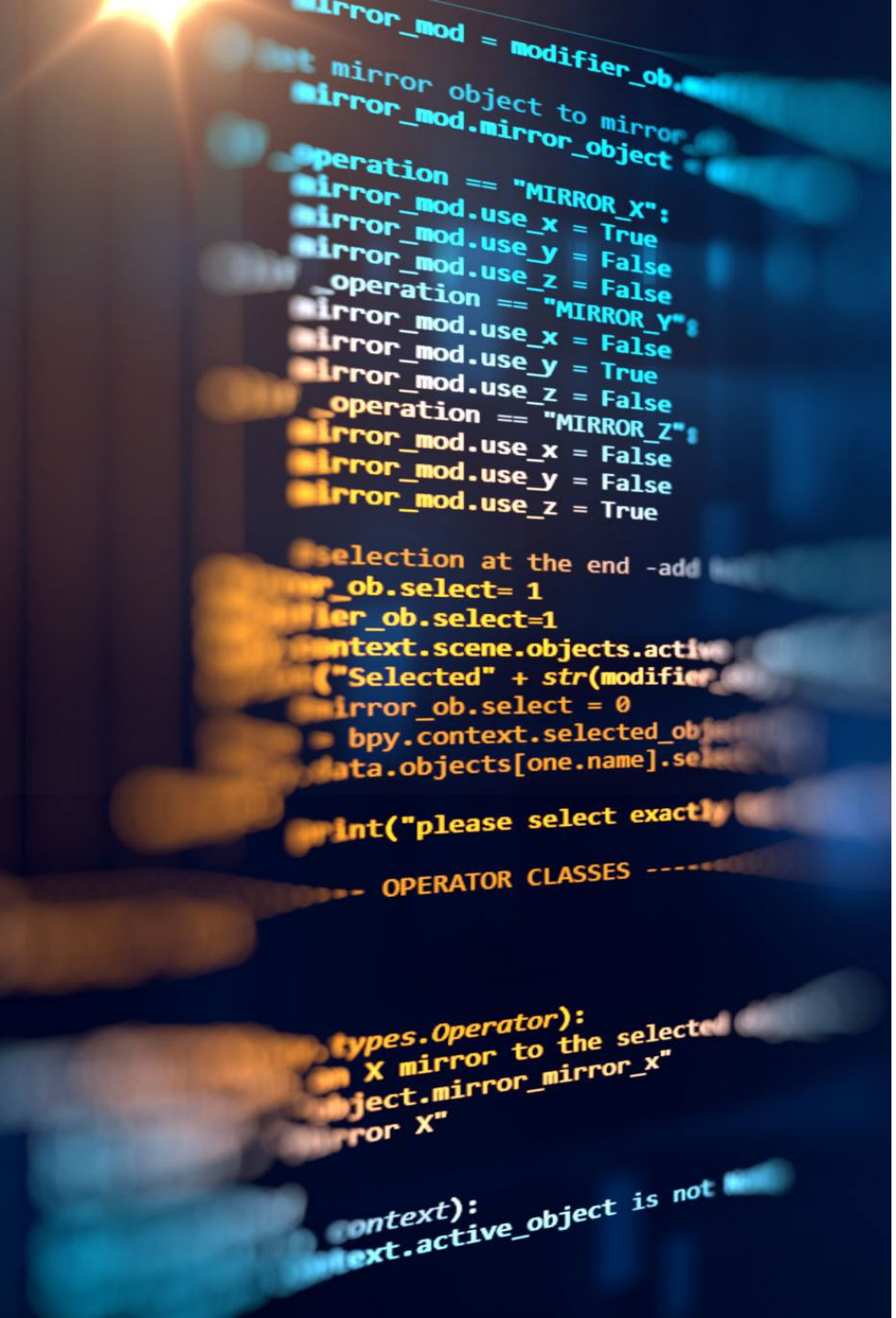


From Waterfall….

… To Agile



- Blackbox testing allows test cases to be built before implementation
- Agile (XP specifically) introduces Test Driven Development as a solution to more testing confidence and motivation



What is Test Driven Development (TDD)

- Test is written ***before*** the class to be tested, and the developer writes unit testing code for nearly *all* production code.
- Write test code
  - Code that fulfills requirements
- Write functional code
  - Working code that fulfills requirements
- Refactor
  - Clean working code that fulfills requirements

TDD Basics – Unit Testing

Red, Green, Refactor



Make it Fail

- No code without a failing test



Make it Work

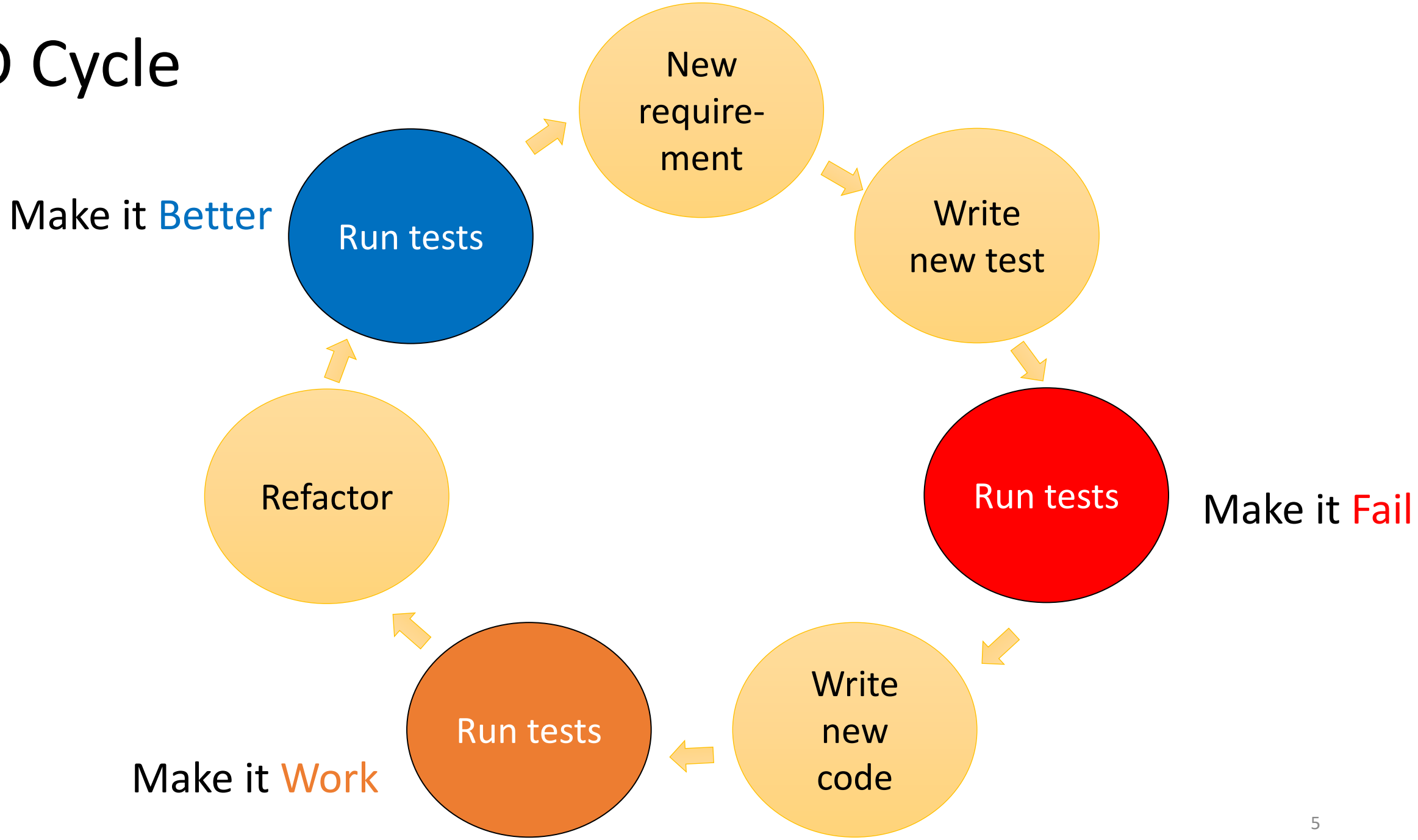
- As simply as possible



Make it Better

- Refactor

TDD Cycle


New require-ment
Write new test
Run tests
Make it Fail
Write new code
Run tests
Make it Work
Refactor
Run tests
Make it Better

Why TDD?

- Imposes developers' discipline
- Provides incremental specification
- Avoid regression errors
- Allows for changing with confidence

TDD + AI: Red → Green → Refactor

1 RED

Write a small test from the requirement.

Run it and confirm a meaningful failure.

2 GREEN

Ask AI to implement the behavior.

Run tests; inspect the implementation.

3 REFACTOR

Improve code structure without changing behavior.

Review diff and rerun tests.

Keep the expected behavior independent of the implementation.

TDD Example

Consider writing a program to score the game of bowling

You might start with the following test

```
public class TestGame extends TestCase {
        public void testOneThrow() {
        Game g = new Game();
        g.addThrow(5);
        assertEquals(5, g.getScore());
        }
}
```

When you compile this program, the test “fails” because..

the Game class does not yet exist.

But:

You have defined two methods on the class that you want to use

https://home.cs.colorado.edu/~kena/classes/5828/f16/lectures/22-tdd.pdf

TDD Example

Now you would write the Game class

```
public class Game {
        public void addThrow(int pins) {
        }
        public int getScore() {
                return 0;
        }
}
```

The **code now compiles but the test will still fail**: getScore() returns 0 not 5

- In Test-Driven Design, we take small, simple steps

- So, we get the test case to compile before we get it to pass

TDD Example

Once we confirm that the test still fails, we would then write the simplest code to make the test case pass; that would be

```
public class Game {
        public void addThrow(int pins) {
        }
        public int getScore() {
                return 5;
        }
}
```

The test case now passes

But this test case is not very helpful

TDD Example

Let's add a new test case to enable progress

```
public class TestGame extends TestCase {
        public void testOneThrow() {
                Game g = new Game();
                g.addThrow(5);
                assertEquals(5, g.getScore());
        }
…
```

```
…
        public void testTwoThrows() {
                Game g = new Game();
                g.addThrow(5);
                g.addThrow(4);
                assertEquals(9, g.getScore());
        }
}
```

The first test passes, but the second case fails (since 9 ≠ 5)

TDD Example

- We have duplication of information between the first test and the Game Class
  - In particular, the number 5 appears in both places
- This duplication occurred because we were writing the simplest code to make the test pass
  - Now, in the presence of the second test case, this duplication does more harm than good
- So, we must now **refactor** the code to remove this duplication

TDD Example

```
public class Game {
        private int score = 0;
        public void addThrow(int pins) {
                score += pins;
        }
        public int getScore() {
                return score;
        }
}
```

Both tests pass now.

Progress!

TDD Example

But now, to make additional progress, we add another test case to the TestGame class

```
public void testSimpleSpare() {
        Game g = new Game()
        g.addThrow(3); g.addThrow(7); g.addThrow(3);
        assertEquals(13, g.scoreForFrame(1));
        assertEquals(16, g.getScore());
}
```

We're back to the code not compiling due to scoreForFrame()

- We'll need to add a method body for this method and give it the simplest implementation that will make all three of our tests cases pass

When AI Writes Both the Code and the Tests

A passing test suite can encode the same misunderstanding as the code.

Input: REGULAR customer · $75 subtotal · SAVE10

Specification

SAVE10 applies at $75.

Discount = $7.50

Final total = $67.50

Hypothetical AI mistake

Applies discount only above $75.

Discount = $0.00

Test expects $75.00

Code and tests can both pass while violating the requirement.

TDD Quiz: From Red to Green



A library system is implementing a Book class. Calling borrow() should mark a book as unavailable.

Current Book class

```
public class Book {
  private boolean isAvailable = true;
  public void borrow() { }
  public boolean isAvailable() {
    return isAvailable;
  }
}
```

JUnit test written first

```
@Test
void testBorrow() {
  Book book = new Book();
  book.borrow();
  assertFalse(book.isAvailable());
}
```

The test compiles but fails. What is the smallest change to make it pass?

Answer: In borrow(), set this.isAvailable = false;

TDD in the Age of AI-Assisted Development

Why a decades-old practice matters again

Traditional development

Developers write tests and most implementation code.

Test-first work can feel like added up-front effort.

AI-assisted development

AI can generate implementation quickly.

Defining correct behavior and reviewing generated code remain essential.

Tests define the target before an AI agent writes the solution.

TDD in the Age of AI-Assisted Development

https://medium.com/@bertha.natalia77/test-driven-development-tdd-is-it-still-relevant-in-2026-76cdce086fcb

TDD is no longer just a manual method for preventing bugs; it is the **control interface for automated software engineering**. As AI models take over code generation, human developers move up the abstraction stack from code writers to system architects. In this environment, the ability to write precise, unambiguous, test-first specifications is the most effective way to ensure that AI-generated software remains robust, maintainable, and aligned with real-world requirements.

Refactoring

What is Refactoring?



Program

Refactored Program

Applying transformations to a program, with the goal of improving its design without changing its functionality

Goal: Keep program readable, understandable, and maintainable. Avoid small problems soon.

Key Feature: Behavior Preserving- make sure the program works after each step; typically small steps

Slide adapted from Alessandro Orso

Behavior Preserving

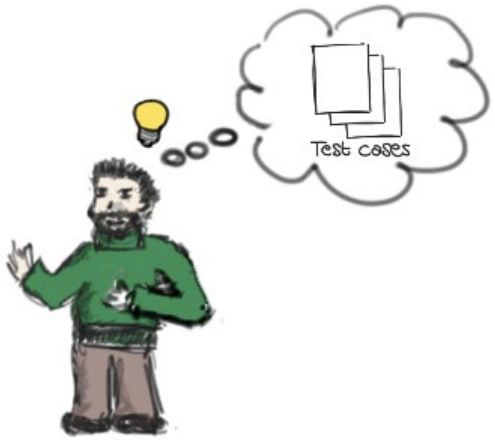


How can we ~~guarantee~~ it?

Test the code

In agile we already have lot of test cases, rerun before and after refactoring)

But beware: No guarantees!



Slide adapted from Alessandro Orso

Behavior Preserving Quiz



Why can’t testing guarantee that a refactoring is behavior preserving?

[ ] Because testing and refactoring are different activities

[✓] Because testing is inherently incomplete

[ ] Because testers are often inexperienced



Program Domain

Test Cases

Why Refactoring?



Requirements Change – different design needed

Design needs to be improved – so that new features can be added; design patterns are often a target

Sloppiness by programmers – copy & paste for a new method

Refactoring often has the effect of making a design more flexible

Have you used Refactoring Before?



Even renaming a class is a refactoring!

(albeit a trivial one)

Many Refactorings in Fowler's Book

- Add parameter
- Change Association
- Reference to Value
- Value to Reference
- Collapse Hierarchy
- Consolidate Conditionals
- Procedures to Objects
- Decompose Conditionals
- Encapsulate Collection
- Encapsulate Downcast
- Encapsulate Field
- Extract Method
- Extract Class
- Inline Class
- Form Template Method
- Hide delegate
- Hide method
- Inline temp

…

Collapse Hierarchy

If a superclass and a subclass are too similar

=> Merge Them



Consolidate Conditional Expression

If there are a set of conditionals with the same results

=> Combine and extract them

```
double disabilityAmount(){
    if (seniority<2)
        return 0;
    if (monthsDisabled>12)
        return 0;
    if (isPartTime)
        return 0;
    // compute disability amount
}
```



Decompose Conditionals

If a conditional statement is particularly complex (can tell what but obscures why)

⇒ Extract methods from conditions, give the right name to the extracted method

⇒ Modify THEN and ELSE part of the conditional

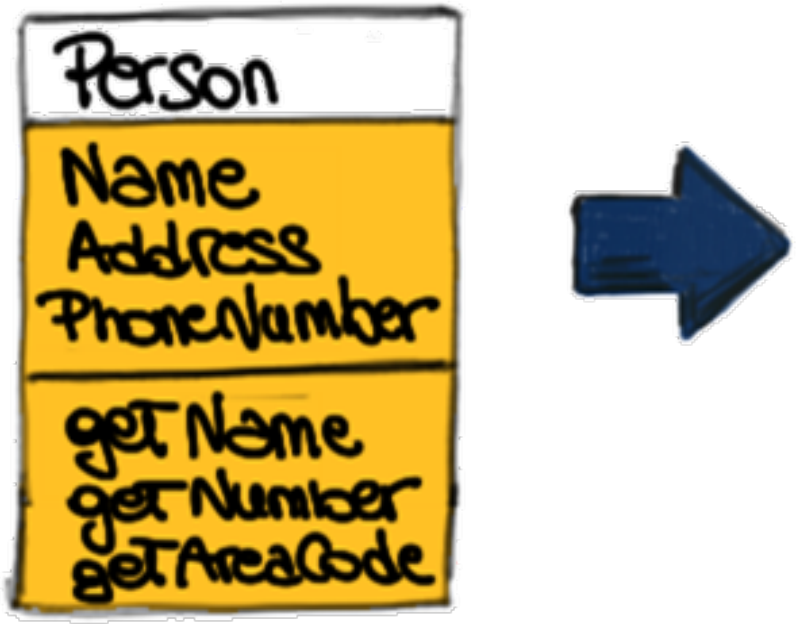
```
if (date.before (SUMMER_START) || date.after (SUMMER_END))
    charge = quantity * winterRate + winterServiceCharge;
else
    charge = quantity * summerRate;
```



Extract Class

If a class is doing the work of two classes

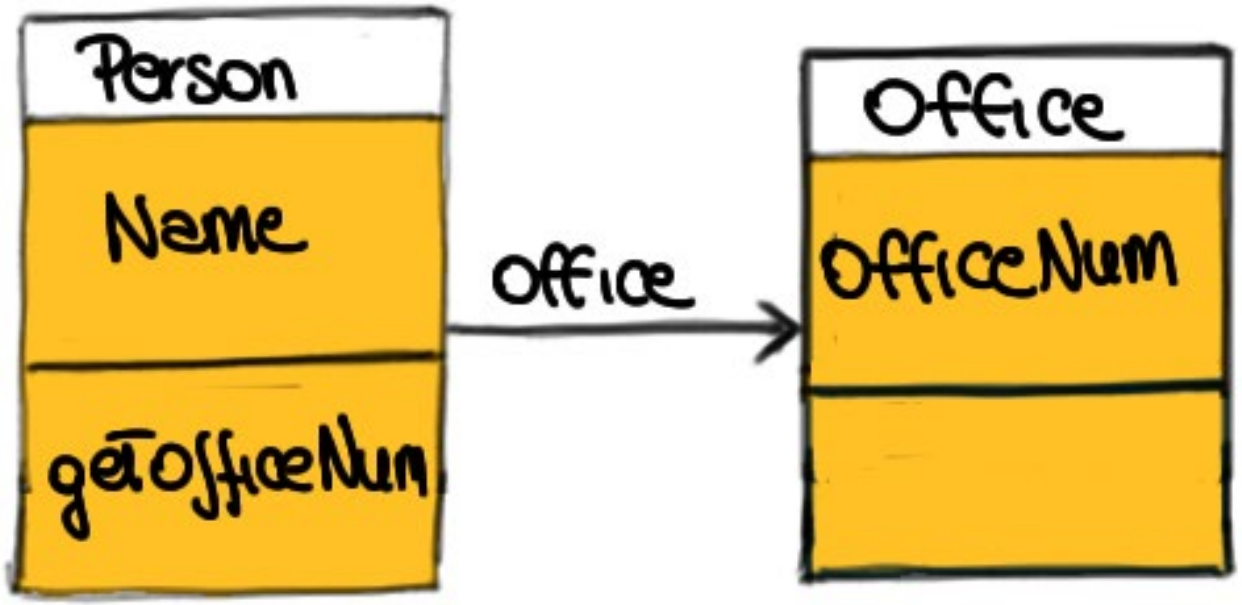
⇒ Create a new class and move the relevant fields/methods (high cohesion, low coupling)



Inline Class

If a class is not doing much during system evolution

$\Rightarrow$ Move its features into another class and delete this one

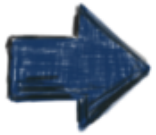


Extract Method

If there is a cohesive code fragment in a large method

=> Create a method using that code fragment, replace code fragment with a call to the method

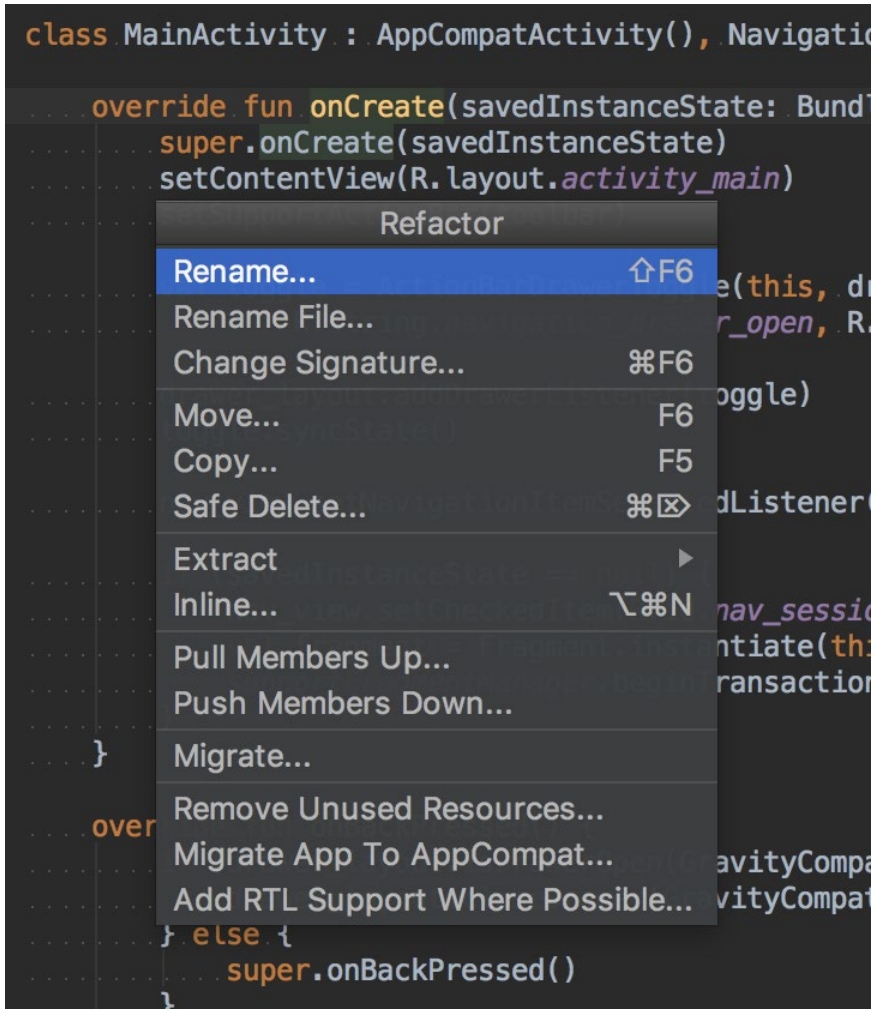
```
void printOwing(){
...
   System.out.printen("name:"+ name+
                      "address:"+address);
    ...
   System.out.printen ("amount owned "+
                       amount)
...
}
```



Refactoring in IDEs

Most IDEs have a set of built-in refactoring tools

The **Refactor** menu includes:
Rename class/method/variable
Change a method signature
Move a class to a new package
Extract a method or variable
Extract a method parameter
Create a new constant
Inline a method
Safe delete



https://medium.com/android-testing-daily/refactoring-28b8a4a07d42

AI-Assisted Refactoring: Opportunities and Risks

AI can help identify or suggest

Long methods and duplicated code

Extract Method / Extract Class

Simplify nested conditionals

Generate regression tests

Engineers must verify

Is there a real code smell?

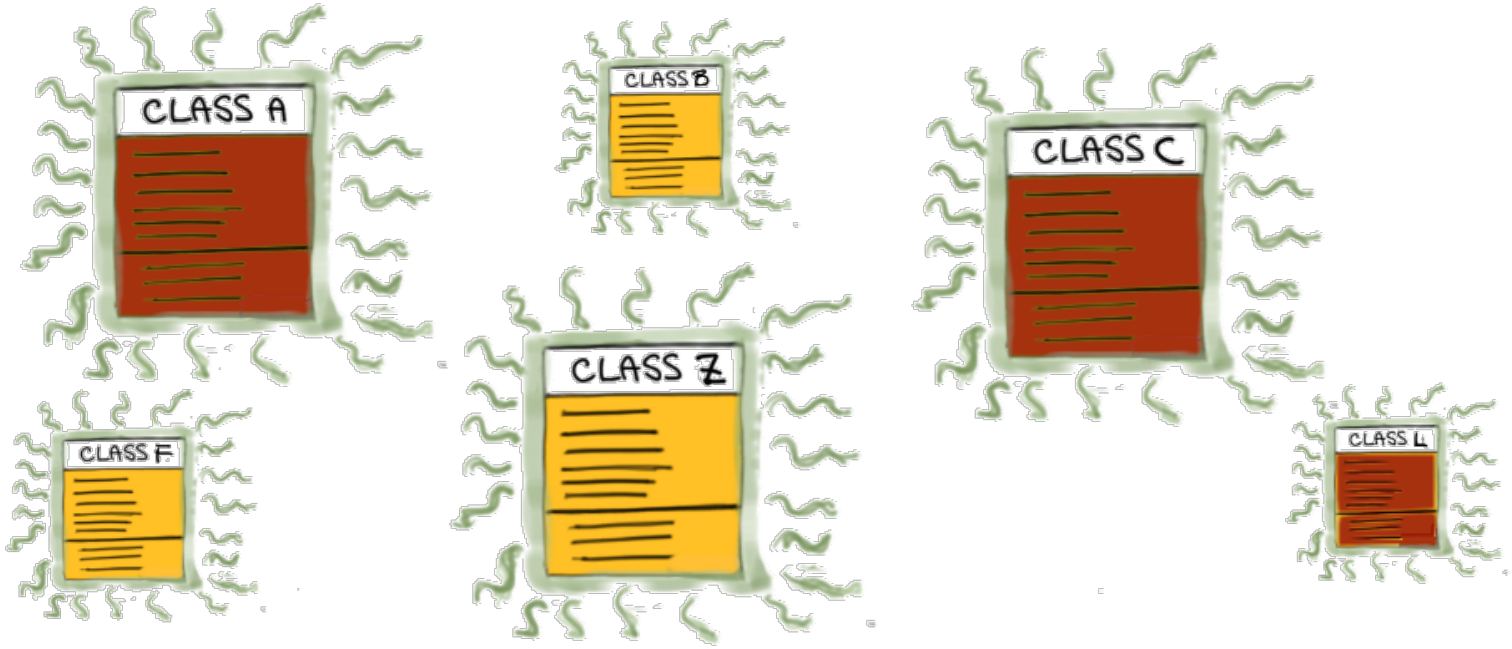
Are responsibilities better separated?

Are all paths preserved?

Do tests check the right behavior?

Refactoring improves internal structure while preserving observable behavior.







- Symptoms that indicate deeper problems in the code.
- Should be able to sense/sniff it.
- Not bugs, indicate weakness in design and hence maintenance in code.

Refactoring Safely with AI

A practical workflow for reviewing AI-generated changes

1. Identify a specific code smell or maintenance problem
2. Run regression tests and establish baseline behavior
3. Ask AI for a small, targeted refactoring
4. Inspect the diff for scope creep or changed behavior
5. Rerun tests and static analysis; accept or revise

Refactoring Example: Checkout Shipping

Scenario: Extract shipping calculation without changing business behavior

Business rule: $7.99 shipping if discounted subtotal < $50; otherwise free.

```
double discountedSubtotal = subtotal -
discount;
double shipping = discountedSubtotal < 50.00
        ? 7.99 : 0.00;
double finalTotal = discountedSubtotal +
shipping;
```

Input

Subtotal: $55.55

Customer: PREMIUM

Membership discount: 10%

Expected: $5.56 discount → $49.99 discounted subtotal → $7.99 shipping → $57.98 final total

AI-Suggested Extract Method: Spot the Defect

The new method looks cleaner, but which amount is passed to it?

```
double shipping = calculateShipping(subtotal);
double finalTotal = subtotal - discount +
shipping;

private double calculateShipping(double amount)
{
    return amount < 50.00 ? 7.99 : 0.00;
}
```

Input: PREMIUM, $55.55

Discount: $5.56

Original subtotal: $55.55

Discounted subtotal: $49.99

AI passes $55.55 to shipping

Incorrect output: $0 shipping; $49.99 final total (expected: $7.99 shipping; $57.98 final total)

Verify the Refactoring: Test, Inspect, Correct

Use the same boundary input before and after refactoring

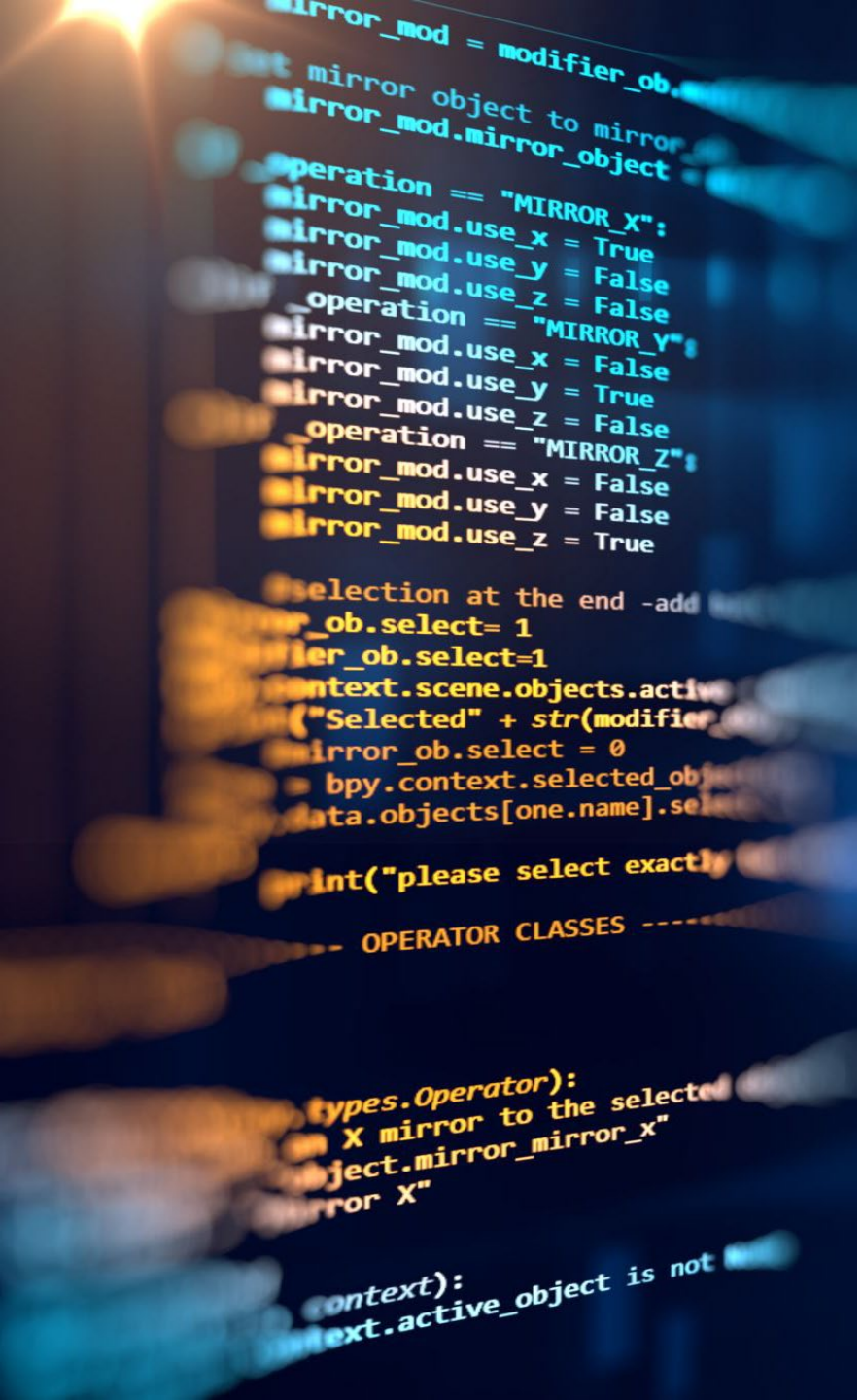
	Original (correct)	AI refactoring (incorrect)
Input	PREMIUM, $55.55	PREMIUM, $55.55
Discount	$5.56	$5.56
Discounted subtotal	$49.99	$49.99
Shipping	$7.99	$0.00
Final total	$57.98	$49.99

```
double shipping = calculateShipping(discountedSubtotal);
```

Industry Practice: Refactoring and Quality Checks

IDE refactoring	IntelliJ IDEA, VS Code / Java extensions	Rename, Extract Method, Move
Static analysis	SonarQube, SonarQube for IDE, SpotBugs	Flag maintainability and defect risks
Regression testing	JUnit, pytest; JaCoCo for coverage	Check behavior after changes
AI coding assistants	Copilot and other LLM-based tools	Suggest targeted refactorings; review diffs

Code review + automated checks + meaningful tests support safe changes.



When to refactor?

- When you find you have to **add a feature** to a program, and the program's code is not structured in a convenient way to add the feature, first refactor the program to make it easy to add the feature, then add the feature.
- During a **code review**: may be the last chance to tidy up the code before it become
- Every step of TDD



When not to Refactor?

- When code is broken (not a way to fix code)
- When a deadline is close
- When there is no reason to!



Quizizz on White Box Testing